Informe de Laboratorio 3

Michelle Sayas. C.I: 31.717.191
y Tulio Cordero. C.I: 30571603

Octubre 2026

1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema con memory-mapped I/O existe un único espacio de direcciones compartido entre la RAM y los registros de los periféricos. En nuestros programas esto se refleja en la sección `.data`, donde cada dispositivo se representa con registros de 32 bits en posiciones consecutivas de memoria: `LuzControl`, `LuzEstado` y `LuzDatos` para el sensor de luminosidad, y `TensionControl`, `TensionEstado`, `TensionSistol` y `TensionDiastol` para el módulo de tensión arterial. Cada uno se declara como `.word 0` y se accede por su dirección, igual que cualquier variable.

En un sistema real, los dispositivos se mapean en regiones altas del espacio de direcciones, separadas de la RAM. En MIPS32 se ubican en la región de kernel no cacheada (`kseg1`, desde `0xA0000000`), y MARS reserva la zona `0xFFFF0000`–`0xFFFFFFFF`. En esta práctica, al no existir hardware real, los registros se simulan en el segmento de datos.

Para `lw` y `sw` esto implica que bastan para toda la E/S. En `ejercicio1_sensor_luz.asm`, `InicializarSensorLuz` usa `sw $t1, 0($t0)` sobre `LuzControl` para enviar la orden de inicialización, y `lw $t1, 0($t0)` sobre `LuzEstado` para consultar el estado. Sin embargo, estas posiciones no se comportan como memoria ordinaria: un valor leído puede cambiar sin intervención del programa y una escritura puede disparar una acción, por lo que no deben almacenarse en caché ni reordenarse sus accesos.

2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

La diferencia principal es que en el memory-mapped I/O los dispositivos comparten el espacio de direcciones con la memoria, mientras que en la E/S por puertos existe un espacio independiente, accesible mediante instrucciones propias (como `IN` y `OUT` en x86) y una señal de bus que distingue memoria de E/S.

El memory-mapped I/O tiene como ventajas que no requiere instrucciones especiales y que permite usar cualquier modo de direccionamiento sobre los registros del dispositivo. Nuestros dos programas lo muestran: toda la comunicación con el sensor y con el módulo de tensión se logró únicamente con `la`, `lw` y `sw`. Como desventajas, consume espacio de direcciones y obliga a evitar el uso de caché en esas regiones. La E/S por puertos, por su parte, no ocupa direcciones de memoria y facilita aislar y proteger la E/S, pero exige instrucciones adicionales y resulta menos flexible.

MIPS32 utiliza principalmente memory-mapped I/O porque es una arquitectura *load/store* con un conjunto de instrucciones reducido y regular, acorde con la filosofía RISC, que evita añadir instrucciones dedicadas a la E/S.

3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos usan direcciones solapadas, ambos responden a un mismo acceso. En una lectura se produce una colisión en el bus de datos que genera valores corruptos, y una escritura puede afectar simultáneamente a ambos dispositivos. En nuestros programas, por ejemplo, si `LuzEstado` y `TensionEstado` ocuparan la misma dirección, la espera activa de un dispositivo podría terminar por el estado del otro, y una escritura de `1` en `LuzControl` iniciaría también una medición de tensión.

El conflicto se evita definiendo un mapa de memoria en el que cada dispositivo tenga un rango exclusivo, y empleando un decodificador de direcciones que active una única señal de selección (*chip select*) por rango. En los programas entregados el ensamblador asigna a cada etiqueta de `.data` una dirección distinta, lo que garantiza que no haya solapes. En sistemas modernos, el firmware y el sistema operativo asignan estos rangos de forma controlada.

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Al reutilizar `lw` y `sw` para acceder a los periféricos, no se necesitan códigos de operación de E/S, modos de direccionamiento propios ni lógica de decodificación adicional dentro del procesador. Esto se aprecia en `controlador_tension`, que inicia la medición con `sw`, espera con `lw` y `beq`, y obtiene los resultados con `lw`, todo con instrucciones comunes del ISA.

Con E/S por puertos serían necesarias instrucciones como `in` (leer de un puerto) y `out` (escribir en un puerto), con variantes por tamaño de dato (byte, media palabra, palabra) y por forma de indicar el puerto (inmediato o en registro). Los `sw` y `lw` de nuestros procedimientos tendrían que sustituirse por estas instrucciones. También podrían requerirse instrucciones de cadena para transferencias en bloque.

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador ejecuta, por ejemplo, `lw $v0, 0($t0)` con `$t0` apuntando a `LuzDatos`, coloca esa dirección en el bus de direcciones y activa la señal de lectura; en un `sw`, como el que escribe en `LuzControl`, además pone el dato en el bus de datos. Un decodificador de direcciones compara los bits más significativos con los rangos asignados. Si la dirección pertenece al rango de un periférico, activa el *chip select* de ese dispositivo y desactiva la RAM.

A continuación, el dispositivo coloca el contenido de su registro en el bus de datos (lectura) o lo captura (escritura). De esta forma el hardware sabe a quién dirigirse únicamente por la dirección, sin que la instrucción lo indique. En MARS este proceso no es visible, porque los registros simulados residen en la memoria de datos.

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En un sistema real normalmente no. Las direcciones de E/S se encuentran en el espacio de kernel, y un acceso desde modo usuario provoca una excepción por error de dirección. En MARS, en cambio, no existe protección: nuestros programas acceden libremente a `LuzControl` o `TensionControl` sin restricción alguna, porque se ejecutan como si tuvieran privilegios totales.

Los mecanismos de protección son: los modos de ejecución usuario/kernel, la unidad de gestión de memoria (MMU) con tablas de páginas y bits de permisos, que solo mapean un dispositivo en el espacio de un proceso si el sistema operativo lo autoriza, y el uso de controladores (*drivers*) a los que los programas acceden mediante llamadas al sistema.

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Nuestros programas usan espera activa: el bucle `esperaInit` en `InicializarSensorLuz` y el bucle `espera` en `controlador_tension` consultan repetidamente el registro de estado, consumiendo ciclos de CPU sin hacer trabajo útil. Para evitarlo existen varias técnicas. Las interrupciones permiten que el dispositivo avise al procesador cuando está listo, liberándolo para otras tareas. El acceso directo a memoria (DMA) transfiere bloques de datos sin intervención de la CPU. También se puede realizar un sondeo periódico controlado por un temporizador, ejecutando otro trabajo entre consultas.

Además, el sistema operativo puede bloquear el hilo o proceso solicitante hasta que ocurra el evento, y es recomendable usar esperas con tiempo límite (*timeout*) para evitar bloqueos indefinidos, algo que nuestros bucles no incluyen.

8. Análisis y Discusión de los Resultados

Ejercicio 1: sensor de luminosidad

En el archivo `ejercicio1_sensor_luz.asm` se definieron en la sección de datos los registros `LuzControl`, `LuzEstado` y `LuzDatos`. El procedimiento `InicializarSensorLuz` escribe 1 en `LuzControl` y realiza una espera activa sobre `LuzEstado` hasta que vale 1. El procedimiento `LeerLuminosidad` devuelve en `$v0` la lectura y en `$v1` el código de estado (0 si la lectura es correcta, -1 si hay error).

```
InicializarSensorLuz:
        la   $t0, LuzControl
        li   $t1, 1
        sw   $t1, 0($t0)          # inicializa el sensor
        la   $t0, LuzEstado
esperaInit:
        lw   $t1, 0($t0)
        li   $t2, 1
        bne  $t1, $t2, esperaInit     # espera activa
        jr   $ra

LeerLuminosidad:
        la   $t0, LuzEstado
        lw   $t1, 0($t0)
        li   $t2, -1
        beq  $t1, $t2, lecturaError
        li   $t2, 1
        bne  $t1, $t2, LeerLuminosidad
        la   $t0, LuzDatos
        lw   $v0, 0($t0)
        li   $v1, 0
        jr   $ra
lecturaError:
        li   $v0, 0
        li   $v1, -1
        jr   $ra
```

Como MARS no dispone de hardware real, el programa principal simula el dispositivo escribiendo directamente en `LuzEstado` y `LuzDatos` antes de cada llamada. Con el dato 742 y estado 1 se imprime la lectura con código de estado 0, y con el estado -1 se informa el error de hardware. Sin dejar `LuzEstado` en 1 antes de inicializar, la espera activa nunca terminaría, lo que evidencia el principal inconveniente del sondeo.

Ejercicio 2: módulo de tensión arterial

En el archivo `ejercicio2_tension.asm` se definieron los registros `TensionControl`, `TensionEstado`, `TensionSistol` y `TensionDiastol`. El procedimiento `controlador_tension` escribe 1 en `TensionControl` para iniciar la medición, espera mientras `TensionEstado` sea 0 y, cuando vale 1, retorna la tensión sistólica en `$v0` y la diastólica en `$v1`.

```
controlador_tension:
        la   $t0, TensionControl
        li   $t1, 1
        sw   $t1, 0($t0)              # inicia la medicion
        la   $t0, TensionEstado
espera:
        lw   $t1, 0($t0)
        beq  $t1, $zero, espera       # 0 -> midiendo
        la   $t0, TensionSistol
```

```
10          lw   $v0, 0($t0)
11          la   $t0, TensionDiastol
12          lw   $v1, 0($t0)
13          jr   $ra
```

El programa principal simula el dispositivo cargando 120 en `TensionSistol`, 80 en `TensionDiastol` y 1 en `TensionEstado`, por lo que la espera termina de inmediato y la salida muestra `Tension sistolica: 120` y `Tension diastolica: 80`.

Discusión general

Ambos ejercicios demuestran que el memory-mapped I/O permite manejar los dispositivos con las instrucciones habituales `lw` y `sw`, tratando sus registros como posiciones de memoria. Se requiere, no obstante, sincronización mediante los registros de estado (`LuzEstado` y `TensionEstado`). La limitación más importante es el uso de espera activa, que desperdicia ciclos de CPU y podría bloquear el programa si el dispositivo nunca responde; esto podría mejorarse con interrupciones o tiempos límite. Para dispositivos simples el modelo es eficaz, pero en sistemas complejos se necesitan capas de abstracción (controladores) que gestionen concurrencia y errores.